

□ How to arise data inconsistency in concurrent access?

Data inconsistency in concurrent access occurs when multiple processes or threads simultaneously interact with shared data without proper synchronization. This can lead to issues like race conditions where the outcome of the execution depends on the particular order. For example two threads attempt to increment a shared counter at the same time, they may both read the current value, increment it, and write it back, resulting only one increment being recorded instead of two.

□ What is critical-section problem?

Let us consider a system consisting of n processes $\{P_0, P_1, P_2, \dots, P_{n-1}\}$. Each process has a segment of code, in which the process may be accessing and updating data that is shared with at least one other process, is called critical section.

Q Explain the requirement which must be satisfied to solve the critical section problem.

critical section problem's ^{solution} must satisfy the following three requirements, these are

1. Mutual Exclusion.

2. Progress.

3. Bounded waiting.

I could describe these requirements in the following manner -

1. Mutual Exclusion: $\rightarrow$ If process P_i is executing in its critical section, then no other processes can be executing in their critical section.

2. Progress $\rightarrow$ If no other processes is executing in its critical section and some process which to enter their critical section, then only those processes not executing their remainder sections, can participate in deciding which will enter its critical section next, and this selection can not be postponed indefinitely.

3. Bounded waiting $\rightarrow$ There exists a bound, or limit on the number of times that other processes are allowed to enter their critical section after a process has made a request to enter its critical section and before that request is granted.

▣ Mention the limitation of Peterson's solution—.

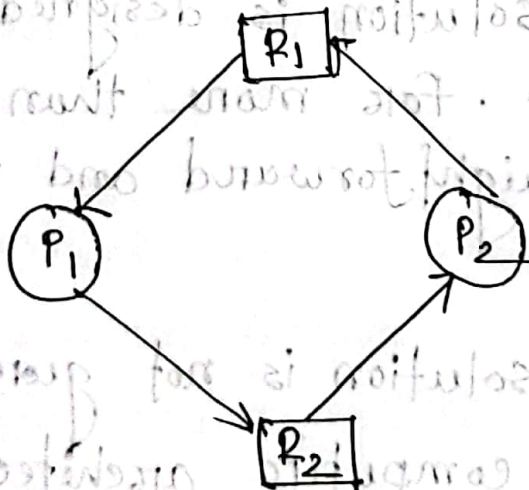
Peterson's solution is a classical algorithm used to address the critical section problem in concurrent programming for two processes. There are some limitations of Peterson's solution that can be pointed out in the following manner $\rightarrow$

1. Peterson's solution is designed specially for two processes. For more than two processes, it is not straightforward and requires additional complexity.
2. Peterson's solution is not guaranteed to work on modern computer architectures.
3. The algorithm uses busy-waiting, where a process continuously checks a condition in a loop until it can enter the critical section.

4. In multi-core systems, peterson's solution may fail due to memory consistency issue.
5. Does not give guarantee fairness.

Define liveness failure in the process execution life cycle —

Liveness failure = A process waiting indefinitely for another process to complete its execution is called liveness failure.



Here, P_1 and P_2 are two processes and resource R_1 and R_2 are allocated correspondingly. But P_1 wants R_2 for execution and

P_2 also require R_1 for execution. This situation are known as liveness failure. There are two types of liveness failure these are

1. Deadlock.
2. Priority Inversion.

1. Deadlock → Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resources acquire by some other process.

2. Priority Inversion → when a higher priority process needs to read or modify kernel data, but a lower priority process is accessing the kernel resource, the higher priority process will have to wait for a low-priority one to finish with the resources.

① What is Semaphore?

Semaphore is an integer variable which is used in mutual exclusive manner by various concurrent co-operative process in order to achieve synchronization.

② Explain Counting Semaphore.

Counting Semaphore range is $-\infty$ to $+\infty$.

Counting Semaphore is a synchronization tool that is used in OS to control the access to shared resources. It is a type of semaphore that allows more than two processes to access the shared resources at the same time. A

Counting Semaphore is represented by an integer value that can be incremented or decremented by the processes.

typedef struct {
int value;

struct process *list;

} semaphore

• Wait (Semaphore *s) {

s → value --;

if (s → value < 0) {

s → list; add this process to s → list;
block();

}

}

• Signal (Semaphore *s) {

s → value ++;

if (s → value <= 0) {

remove a process p from s → list;
wake up (p);

}

}

③ Difference between deadlock and starvation.

Deadlock	Starvation
① When two or more processes can try to get the same multiple resources at a same time, called deadlock.	① High priority processes keep executing and low priority processes are blocked, called starvation.
② Resources are blocked by the processes.	② Resources are continuously utilized by high priority processes.
③ Mutual exclusion, Hold & wait, NO preemption	③ priorities are assigned to the processes.
④ Also known as circular wait	④ Also known as lived lock
⑤ It can prevent by avoiding the necessary conditions for deadlock.	⑤ It can be prevented by Aging

④ What are Mutex locks?

A mutex (mutual exclusion) lock makes it possible to implement mutual exclusion by limiting the number of threads or processes that can simultaneously acquire lock.

⑤ State the solution to the critical section problem using mutex locks.

do {

acquire lock

critical section

release lock

remainder section

} while (true)

0 $\rightarrow$ available

acquire() {
 while (!available);
 available = false;
}

release() {
 available = true;
}

⑥ How to solve the critical section problem using peterson's solution.

do {
 Flag[i] = true;
 turn = i;
 while (flag[j] && turn == j)

Critical section

Flag[i] = false;

Remainder section

} while (true)

do {
 Flag[j] = False
 turn = i;

while (flag[i] && turn == i)

Critical section

flag[j] = False

Remainder section

} while (true)

⑦ Why Synchronization tools are needed to ensure concurrent access?

Synchronization tools are crucial in concurrent programming because they help manage the access to shared resources, ensuring that multiple processes can operate correctly and without interference.

Main reason why tools are necessary

1. Preventing Data corruption.
2. Avoiding Race Conditions.
3. Ensuring consistent state.
4. Managing Deadlock and starvation.
5. Coordinating Task execution.
6. Supporting parallelism.

Example:

- Mutex
- Semaphores
- Monitors
- Peterson's solution.